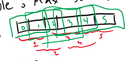
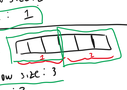
fixed size sliding window: a method used to efficiently solve problems that involve defining a window or range in the input data (array or string) and then moving that window across the data to perform some operation within the window.

Example: Max sum of 2 contiguous elements in array.

arr: 

* window size: 2
* slide: 1

arr: 

* window size: 3
* slide: 3

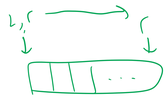
More interesting problem:

Dynamic sliding window: same idea as above, but the window size is variable. You want to use this when you are searching for some contiguous property, but are unsure what the optimal window size will be.

Example: leetcode question fruit in baskets.

Dynamic sliding window optimizes the size of the window, by only considering window sizes of length greater than the current window size, starting from a different index.

* In general, it takes $O(n^2)$ time to iterate through all windows, and dynamic sliding window brings this down to $O(n)$.

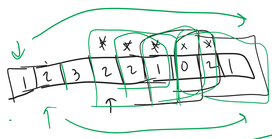


compute all windows
Brute force $O(n^2)$

Dynamic sliding window uses two pointers:

* l: to decrement window size on a condition
* r: to increase window size based on a condition.

num_fruits = 4

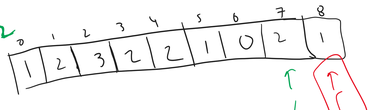


l: 1 2

r: 1 2 3

```
for i in range(len(fruits)):
    for j in range(i, len(fruits)):
```

$O(n^2)$



l: len(num_fruits) > 2

l += 1

r: else: r += 1

num_fruits:

{ 1: 1

2: 1

l → n (n size)

r → n (n size)

r - l + 1 = window size

ans = 4

2n = $O(n)$